

PREMETADATED

Vintage scientific illustration for Typst



Version 0.1.0

github.com/baptiste/premetadated

Contents

1. Overview	3
1.1. Importing	3
2. The drawing model	3
3. Elliptical-nib strokes	3
3.1. Variable pens	4
3.2. Dashes and pressure	4
4. Three-dimensional geometry	4
4.1. Primitives	5
4.2. Parametric tubes	5
5. Surface styles and light	5
6. Illustration helpers	6
7. Optics and mechanics	6
8. Document styles	7
9. Examples and validation	7
10. Credits and licensing	7

1. Overview

Premetadated is a drawing toolkit for historical-looking scientific figures. It combines Typst and Cetz for page composition with a bundled Rust/WASM engine for elliptical-pen envelopes, 3D projection, hidden-line removal, and robust solid geometry. Output remains vector-based: labels are Typst text and marks are Cetz paths rather than raster effects.

The package entrypoint exports six namespaces: `stroke`, `geometry`, `illustration`, `optics`, `mechanics`, and `style`. Most documents need only the `geometry` and `illustration` namespaces.

1.1. Importing

From a repository checkout:

```
#import "lib.typ" as premetadated
#let geo = premetadated.geometry
#let draw = premetadated.illustration
```

For a future Typst Universe release, use:

```
#import "@preview/premetadated:0.1.0"
```

The checked-in WASM plugin is loaded internally. Users do not need Rust unless they modify the geometry engine.

2. The drawing model

Two representations meet in the package:

- A 2D path is an array of cubic Bézier segments. Each segment is (P_0, P_1, P_2, P_3) , where every point is an (x, y) pair.
- A 3D scene is a list of shape dictionaries. `geometry.render` projects their visible paths and applies the same elliptical nib to each polyline.

Drawing helpers are intended to run inside `illustration.canvas`, whose default unit is one centimetre.



A cubic path rendered as an elliptical-nib rule.

```
#draw.canvas({
  draw.engraved(draw.line-path((0, 0), (4, 1)))
})
```

3. Elliptical-nib strokes

`stroke.nib-stroke` sweeps an ellipse along a cubic path and fills the resulting envelope. A constant pen is (major-axis, minor-axis, angle).



```
stroke.nib-stroke(
  (((0, 0), (1, 1.25), (2.3, -1), (3.5, 0.15))),
  pen: (0.30, 0.075, 24deg),
  fill: black,
)
```

Use `closed: true` for a closed centreline. `epsilon` controls Bézier flattening; smaller values produce a finer envelope. `stroke.envelope` returns polygon data without drawing it.

3.1. Variable pens

Explicit samples interpolate axes and angle by arclength:



Explicit interpolation



Tangent-following calligraphy

```
pen: (  
  mode: "explicit",  
  samples: (  
    (arclength: 0, a: .32, b: .06, angle: 15deg),  
    (arclength: 4, a: .16, b: .04, angle: 65deg),  
  ),  
)
```

Calligraphic samples derive orientation from the path tangent:

```
pen: (  
  mode: "calligraphic",  
  offset: 20deg,  
  samples: ((arclength: 0, a: .28, b: .07)),  
)
```

3.2. Dashes and pressure

A dash dictionary accepts lengths, offset, jitter, and seed. Pressure clamps the smallest printable nib axis and represents lighter requests with a deterministic broken duty cycle.



Seeded irregular dashes



Pressure taper breaking into marks

```
dash: (lengths: (0.32, 0.16), jitter: 0.04, seed: 7),  
pressure: (minimum-axis: 0.028, period: 0.32, seed: 17),
```

4. Three-dimensional geometry

Construct shapes, then pass them positionally to `geometry.render`.



Perspective camera



Low transverse camera

```
#draw.canvas({  
  geo.render(  
    geo.sphere((0, 0, 0), 1),  
    geo.cylinder(0.45, (-1.8, 0, -0.8), (-1.8, 0, 0.8)),  
    eye: (4, 5, 3),  
    center: (0, 0, 0),  
    width: 6,  
    height: 5,  
    pen: (0.025, 0.007, 25deg),  
  )  
})
```

The camera parameters are eye, center, up, width, height, fovy, near, far, and visibility sampling step. Use `geometry.paths` instead of `render` when projected polylines are needed without nib rendering.

4.1. Primitives

- `sphere(center, radius)` and `ellipsoid(center, radii)`
- `cube(min, max)` and `rounded-box(min, max, radius:)`
- `cylinder(radius, start, end)` and `cone(radius, base, apex)`
- `torus(center, major-radius, minor-radius)`
- `rounded-polyhedron(vertices, radius:, detail:)`
- `tube(points, radius:, sides:, closed:)`
- `tube-curve(function, start:, end:, samples:, radius:, sides:, closed:)`

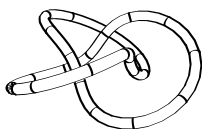
Rounded polyhedra approximate the convex Minkowski sum of their input vertices with a sphere. Their original convex-hull edges receive dedicated transition bands so rounding remains legible without exposing triangulation diagonals.



Sphere, ellipsoid, cube, cylinder, cone, torus, and rounded box.

4.2. Parametric tubes

```
#let tau = 2 * calc.pi
#let trefoil(t) = (
  (2 + calc.cos(3 * t)) * calc.cos(2 * t),
  (2 + calc.cos(3 * t)) * calc.sin(2 * t),
  calc.sin(3 * t),
)
#let knot = geo.tube-curve(
  trefoil,
  start: 0,
  end: tau,
  samples: 120,
  radius: 0.16,
  sides: 14,
  closed: true,
)
```



A closed trefoil tube with hidden crossings removed.

5. Surface styles and light

Every shape accepts a pattern. General solid patterns are `geometry.texture.outline`, `striped`, and `lit-hatch`. Spheres and ellipsoids also support latitude/longitude grids, random equators, random circles, and lighting-aware stippling.

```
pattern: (geo.texture.lit-hatch)(
  light: (-1.0, -0.5, 1.0),
  count: 560,
  length: 0.18,
  crosshatch: 0.72,
  seed: 11,
)
```

`count` is the number of candidate marks. `crosshatch` is the darkness threshold for a second direction. All pseudo-random patterns are deterministic for a given seed.



Latitude / longitude



Lit hatch



Lit stipple

```
pattern: (geo.texture.lit-stipple)(
  light: (-1.0, -0.5, 1.0),
  count: 5200,
  min-size: 0.005,
  max-size: 0.032,
  gamma: 1.3,
  distribution: "fibonacci",
  seed: 11,
)
```

6. Illustration helpers

`illustration.palette` provides paper, ink, and pale-ink. The most useful helpers are:

- `canvas(body, length:)` creates a Cetz canvas.
- `line-path(start, end)` returns a straight cubic path.
- `engraved(path, width:, angle:, dash:, pressure:, epsilon:, ink:)` draws a narrow elliptical-nib rule.
- `label(point, body)` places a paper-backed label.
- `figure-caption(body)` formats a compact figure caption.



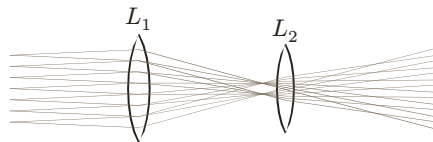
Engraved rule, label, eye, pointing hand, and mirrored raised hand.

The eye and hand helpers currently place vendored Twemoji SVG placeholders. They support origin, direction, and size; hands additionally support pose and handedness. `hand-anchors` returns approximate wrist, palm, finger, and thumb attachment coordinates for vector annotations.

7. Optics and mechanics

The standalone optics module has no drawing dependency. Sources are created with `ray`, `point-source`, or `beam-source`; elements with `ideal-lens`, `mirror`, `blocker`, `aperture`, or `beam-splitter`. `trace-ray` and `trace-scene` return point arrays and preserve brightness and wavelength metadata. Splitters create independent transmitted and reflected branches.

At a lens the local paraxial slope follows $u' = u - \frac{h}{f}$. Mirrors use $\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$. The tracer always selects the nearest finite-segment intersection, while blockers and the opaque leaves of an aperture terminate the ray.



A finite beam traced through two ideal thin lenses.

`import-ray-optics(json-data, beam-rays:, angle-rays:)` accepts the linear `Beam`, `SingleRay`, `PointSource`, `IdealLens`, `Mirror`, `BeamSplitter`, `Blocker`, `Aperture`, and `CropBox` objects exported by Ray Optics Simulation. Unsupported curved and refractive-volume objects are ignored. The complete `examples/kohler.typ` plate reads `kohler.json`, computes every ray, and only then converts traces with `ray-path` for engraving. `lens-path` remains a visual outline helper and does not participate in the calculation.

The mechanics namespace provides `rod`, `collar`, `joint`, and `lit-joint` constructors. These return geometry primitives suitable for the same scene and camera pipeline as all other solids. See `examples/governor-patent.typ` for a complete assembly.



Rods, collar, and lighting-aware joints in one assembly.

8. Document styles

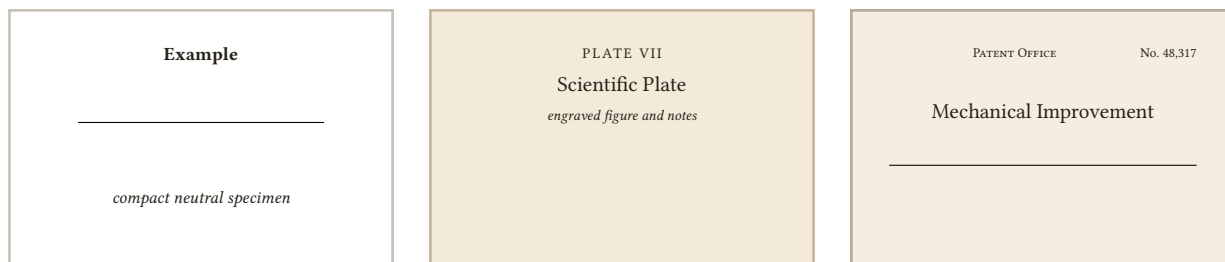
Three show-rule templates provide restrained page furniture:

```
#show: premetadated.style.example.with(width: 16cm, height: 10cm)
```

```
#show: premetadated.style.plate.with(  
  number: [Plate VII],  
  title: [The Compound Microscope],  
  subtitle: [longitudinal section and principal rays],  
)
```

```
#show: premetadated.style.patent.with(  
  title: [Improvement in Rotary Governors],  
)
```

All accept page, paper, ink, and margin overrides. Keep reusable geometry in a namespace module and page-specific layout in the document using the template.



Miniatures of the example, plate, and patent page vocabularies.

9. Examples and validation

The `examples/` directory contains focused stroke, geometry, shading, moon, solid, knot, gesture, optics, and mechanical plates. Compile nested documents with the repository as project root:

```
typst compile --root . examples/solids.typ examples/rendered/solids.pdf  
typst compile --root . manual.typ manual.pdf
```

Contributors changing the Rust engine should run:

```
cargo fmt --check  
cargo clippy --all-targets --all-features -- -D warnings  
cargo test --all-features  
cargo build --release --target wasm32-unknown-unknown --features typst-plugin
```

10. Credits and licensing

Premetadated directly depends on Cetz and Larnt; Larnt is a Rust rewrite of Michael Fogleman's `ln`. MetaPost supplied the elliptical-pen model, while Fiziko and vintage-latex inspired much of the visual vocabulary and specimen subject matter. The eye and hand placeholders are Twemoji graphics.

See `ACKNOWLEDGMENTS.md` for detailed links, roles, and third-party licenses. Premetadated itself is distributed under the Mozilla Public License 2.0.